IdPGroup

Identification functions for different p-groups

1.0

11 August 2026

Henrik Schanze

Henrik Schanze Email: henrik.schanze@tu-braunschweig.de
Homepage: https://www.iaa.tu-bs.de/henschan
Address: Universitaetsplatz 2

Contents

1 Introduction **3**

2 Functionality **4**
2.1 Identifying groups via a search tree 4
2.2 Loading (strong) siblings 6
2.3 Functions for calculating (strong) siblings 8
2.4 Functions for building a search tree 9

References **14**

Index **15**

Chapter 1

Introduction

The purpose of the IdPGroup package is to implement identification functions for different p-groups and thereby extend the functionality of the SmallGrps library. Currently, this includes an identification function for groups of order 512, as described in [ES26].

Details of the implementation and functionality of the identification functions are discussed in section 2.1. Section 2.2 introduces the definitions of (strong) siblings and includes functions to load all such sets of groups of certain order. Functions that were used to calculate these lists of groups and which can be used to calculate them for different orders can be found in section 2.3. Finally, section 2.4 describes all functions that were used to build the search tree in the described identification routine and which can be used to build other such trees.

Chapter 2

Functionality

This chapter lists all functions contained in the IdPGroup package.

2.1 Identifying groups via a search tree

This section briefly describes the functions and variables of this package associated with the identification of an arbitrary group of order 512 via a search tree. For further informations see [ES26].

By 'identify' a group G, we mean that we want to find an *id* such that `SmallGroup(512, id)` is isomorphic to G. This is done by calculating invariants and comparing them with the ones of the remaining candidates. This process is organized in the tree `ID_GROUP_512_TREE` (2.1.3). The tree used is implemented as nested records with the following components:

- "fp": a list of integers, each being the encoded fingerprint of groups resulting from the specific test performed at the current level of the identification routine.
- "next": a list comprised of the respective subbranches corresponding to the fingerprints in "fp". This is either a list of ids or a branch implemented as a record itself.

Depending on the depth in the tree, each record also has one of the following two components:

- "test": a string that indicates which test was used via `TestOnGroup` (2.4.3) to create the listed fingerprints.
- "desc": a list of integers that encode a description of the performed powermap-test.

The first levels of `ID_GROUP_512_TREE` (2.1.3) try to distinguish groups by the following invariants:

- the rank of the group,
- the collected orders of all elements of the group,
- the numbers of conjugacy classes of each length and representative order (the order of the elements in the class) of the group,
- the isomorphism type of the parent of the group, i.e. the quotient of the group and the smallest non-trivial element of the lower-p-central series,
- the abelian invariants, i.e. the isomorphism types of the factor groups of the derived series of the group G and

- the cycle structure of the permutations induced on all clusters of conjugacy classes of same length and representative order by the power map $x \mapsto x^3$.

If necessary, these invariants are followed by invariants resulting from power maps evaluated on partitions of G. The initial partition is made of the conjugacy classes of G grouped such that conjugacy classes of the same length and same representative order belong to the same partitioning subset. The collected result of a power map applied to a subset forms an invariant used to distinguish groups in these levels of the search tree. The describing parameters (e.g. which subset and power map) are determined by the content of the component "desc". After the evaluation, the partition is refined according to the respective result. If the preceding invariants do not lead to a single group, the following (more time consuming) invariants are calculated:

- the isomorphism types of maximal factors formed by factoring out subgroups generated by elements of the smallest non-trivial subgroup in the lower-p-central series,
- the isomorphism types of maximal factors formed by the group and central subgroups and
- the isomorphism types of all maximal subgroups.

After this process, 139 pairs and 1 triplet of groups remain unidentified. These groups are distinguished by a random isomorphism test, the same way ultimately used in the SmallGrps library for the groups of order 256, for more details see [BE99].

2.1.1 InfoId512

▷ `InfoId512` (info class)

Info class for the identification routine of groups of order 512.

The package installs a method for the attribute `IdSmallGroup` (**smallgrp: IdSmallGroup**) (a synonym of `IdGroup` (**smallgrp: IdGroup**)) of the SmallGroups library. For a group G of order 512 this method returns the pair `[p^n,i]` such that G is isomorphic to `SmallGroup(p^n,i)`. For all other groups, the methods of the SmallGroups library are used.

Example

```
gap> G := SmallGroup(512, 606);;
gap> H := Subgroup(G, SmallGeneratingSet(G));;
gap> IdGroup(H);
[512, 606]
gap> G := SmallGroup(512, 1405);;
gap> H := Subgroup(G, SmallGeneratingSet(G));;
gap> IdSmallGroup(H);
[512, 1405]
```

2.1.2 IdGroup512

▷ `IdGroup512(`*`G[, opt]`*`)` (function)

Returns: Integer id such that `SmallGroup(512, id)` is isomorphic to *`G`*.

Identifies the group *`G`* of order 512. This function accepts a record *`opt`* of options with the following content:

- *`path`*: a boolean that determines if the path through the tree shall be recorded while identification and returned as well.

- `tree`: a search tree that was built using the invariants resulting from *tests*.
- `anupq`: a boolean that determines if *IsPqIsomorphicPGroup* from the Anupq-Package is used to find the group isomorphic to `G` among the remaining candidates.

Example

```
gap> G := Image(IsomorphismPermGroup(SmallGroup(512, 1984)));;
gap> IdGroup512(G);
1984
gap> IdGroup512(G, rec(path := true));
[ 1984, rec( fp := [ 2, 8217, 5321 ], pos := [ 2, 188, 1 ],
        tests := [ "Rank", "ElmOrds", [ 40511009 ] ] ) ]
gap> H := Image(IsomorphismPermGroup(SmallGroup(512, 449)));;
gap> IdGroup512(H, rec(anupq := false));
449
gap> IdGroup512(H, rec(anupq := true));
449
```

2.1.3 ID_GROUP_512_TREE

▷ `ID_GROUP_512_TREE` (global variable)

The search tree of groups of order 512 that can determine the vast majority of groups. Only a small part is loaded initlialy. Further branches are loaded if necessary.

2.1.4 LoadTotalTree

▷ `LoadTotalTree()` (function)

Loads the entire search tree `ID_GROUP_512_TREE` (2.1.3).

2.2 Loading (strong) siblings

This section describes the functions used to load precalculated lists of (strong) siblings, i.e. groups satisying the following definitions:

- *subgroup-equivalent groups* are groups with a bijection between the sets of proper subgroups s.t. preimage and image are isomorphic,
- *factor-equivalent groups* are groups with a bijection between the sets of proper factors s.t. preimage and image are isomorphic,
- *siblings* are subgroup- and factor-equivalent groups, where the bijection of proper subgroups preserves conjugacy and maps the Frattini subgroup, lower and upper central series onto the respective counterparts,
- *strong siblings* are siblings that are also Brauer Pairs.

All (strong) siblings of orders dividing 512 together with some trivial cases are implemented in this package, and are callable by the following functions.

2.2.1 Siblings

▷ `Siblings(order, justids[, indices])` (function)

Returns: A list of lists of siblings of size *order*. The parameter *justids* determines wether the ids or the groups (default) shall be returned.

If available, loads and returns a list of siblings of size *order*. Currently available sizes are:

- all orders of the form pq for p and q primes,
- all powers of 2 up to 2^9,
- all powers of 3 up to 3^6,
- all powers of a prime up to p^4 and
- all orders smaller than 256 except 60, 120, 160, 168, 180, 189, 192 and 240.

If no list is available for the provided order, `fail` is returned. If a list *indices* is given, only the respective entries are returned.

Example

```
gap> Siblings(101);
[ ]
gap> Siblings(128);
[ [ <pc group of size 128 with 7 generators>,
    <pc group of size 128 with 7 generators> ],
  [ <pc group of size 128 with 7 generators>,
    <pc group of size 128 with 7 generators> ],
  [ <pc group of size 128 with 7 generators>,
    <pc group of size 128 with 7 generators> ] ]
gap> Siblings(128, true);
[ [ 1317, 1322 ], [ 1327, 1329 ], [ 1597, 1598 ] ]
gap> Siblings(256, true, [1..3]);
[ [ 227, 228 ], [ 258, 260, 261 ], [ 845, 846 ] ]
```

2.2.2 StrongSiblings

▷ `StrongSiblings(order, justids[, indices])` (function)

Returns: A list of lists of strong siblings of *order*.

Loads and returns a list of strong siblings of *order* if available. The parameter *justids* controls if just the ids or the groups (default) shall be returned. Currently available sizes are:

- all orders of the form pq for p and q primes,
- all powers of 2 up to 2^9,
- all powers of 3 up to 3^6,
- all powers of a prime up to p^4 and
- all orders smaller than 256 except 60, 120, 160, 168, 180, 189, 192 and 240.

If no list is available for the provided order `fail` is returned. If a list *indices* is given, only the respective entries are returned.

Example

```
gap> StrongSiblings(256, true);
[ [ 1734, 1735 ], [ 1736, 1737 ], [ 1739, 1740 ], [ 1741, 1742 ],
  [ 3678, 3679 ], [ 4154, 4157 ], [ 4155, 4158 ], [ 4156, 4159 ] ]
gap> StrongSiblings(256, false, [1..3]);
[ [ <pc group of size 256 with 8 generators>,
    <pc group of size 256 with 8 generators> ],
  [ <pc group of size 256 with 8 generators>,
    <pc group of size 256 with 8 generators> ],
  [ <pc group of size 256 with 8 generators>,
    <pc group of size 256 with 8 generators> ] ]
gap> StrongSiblings(256, true, [1..3]);
[ [ 1734, 1735 ], [ 1736, 1737 ], [ 1739, 1740 ] ]
```

2.3 Functions for calculating (strong) siblings

This section describes all functions for calculating (strong) siblings of a given order.

2.3.1 InfoSiblings

▷ `InfoSiblings` (info class)

Info class for the calculation of (strong) siblings.

2.3.2 CalculateSiblings

▷ `CalculateSiblings(`*`order`*`)` (function)

Returns: A list of lists of ids of groups of size *`order`* that are siblings.

Determines all siblings among the groups of size *`order`* by applying the tests `"MaxSubId"`, `"SpSubId"`, `"SubId"`, `"MaxFactId"` and `"FactId"` iteratively. The tests `"MaxSubId"` and `"MaxFactId"` are not necessary, but are efficient and eliminate some sets before applying the more costly tests.

Example

```
gap> CalculateSiblings(128);
[ [ 1317, 1322 ], [ 1327, 1329 ], [ 1597, 1598 ] ]
```

2.3.3 CalculateStrongSiblings

▷ `CalculateStrongSiblings(`*`order[, list]`*`)` (function)

Returns: A list of lists of ids of groups of size *`order`* that are strong siblings.

If no lists of siblings is given via *`list`*, first calculate all siblings of size *`order`* via `CalculateSiblings` (2.3.2), and then determines all strong siblings among them.

Example

```
gap> CalculateStrongSiblings(256);
[ [ 1734, 1735 ], [ 1736, 1737 ], [ 1739, 1740 ], [ 1741, 1742 ], [ 3678, 3679 ],
  [ 4154, 4157 ], [ 4155, 4158 ], [ 4156, 4159 ] ]
gap> CalculateStrongSiblings(256, [[227, 228], [1734, 1735]]);
[ [1734, 1735 ] ]
```

2.4 Functions for building a search tree

This section describes all functions and variables connected to building a search tree that can be used with `IdGroup512` (2.1.2) to identify groups via implemented invariants.

In short, this is done by initializing a tree via calling `TestOnClusterList` (2.4.4) with a selected test on a list of ids of groups of given order and then iterating the following three steps:

- collecting the clusters of the current tree via `ClusterList` (2.4.2),
- performing a chosen test on these clusters via `TestOnClusterList` (2.4.4) and
- extending the tree with these test results via `AddClusterTestResults` (2.4.5).

A very effective set of invariants can be obtained by calling `CocTestOnClusterList` (2.4.7) in the second step. This should only be done when the groups of each provided cluster yield the same value when the test `CocList` is applied. Furthermore these invariants should be used for as few as possible clusters since `CocTestOnClusterList` (2.4.7) includes a search for suitable parameters that can take quite some time.

2.4.1 InfoTreeBuild

▷ `InfoTreeBuild` (info class)

Info class for the building of a search tree.

2.4.2 ClusterList

▷ `ClusterList(`*`tree`*`)` (function)

Returns: A list of all clusters of groups in *`tree`* (i.e. the leaves of *`tree`*).

Iterates through *`tree`* and collects information about each leaf as a record. This includes the size of the leaf, the path through *`tree`* leading to the leaf, the tests performed including descriptions of performed power map tests and the ids of the groups contained in the leaf.

Example

```
gap> tree := rec( fp := [ 63049, 75721, 4310 ],  next := [ [ 3, 8 ], [ 4 ],
gap>            rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ],
gap>                test := "MaxSubId" ) ], test := "ElmOrds" );;
gap> ClusterList(tree);
[ rec( descl := [  ], ids := [ 3, 8 ], path := [ 63049 ], size := 2,
       tests := [ "ElmOrds" ] ),
  rec( descl := [  ], ids := [ 4 ], path := [ 75721 ], size := 1,
       tests := [ "ElmOrds" ] ),
  rec( descl := [  ], ids := [ 5 ], path := [ 4310, 11744 ], size := 1,
       tests := [ "ElmOrds", "MaxSubId" ] ),
  rec( descl := [  ], ids := [ 6, 7 ], path := [ 4310, 86646 ], size := 2,
       tests := [ "ElmOrds", "MaxSubId" ] ) ]
```

2.4.3 TestOnGroup

▷ `TestOnGroup(`*`G, test[, opt]`*`)` (function)

Returns: The result of *`test`* applied to *`G`*, if desired encoded as an integer.

Calculates an invariant of the group *`G`*. The following choices for *`test`* are implemented:

- *"ElmOrds"*: the collected orders of the elements of *G*,
- *"Center"*: the abelian invariants of the center of *G*,
- *"Rank"*: the rank of *G*,
- *"AbelInv"*: the order and abelian invariants of each term in the derived series of *G*,
- *"UCSId"*: the isomorphism types of the proper subgroups in the upper central series of *G* via IdGroup,
- *"LCSId"*: the isomorphism types of the proper subgroups in the lower central series of *G* via IdGroup,
- *"PCSId"*: the isomorphism types of the proper subgroups in the p-central series of *G* via IdGroup,
- *"SpSubId"*: the isomorphismtypes of the Frattini subgroup, the terms in the upper and lower central series of *G* via IdGroup,
- *"MaxSubId"*: the collected isomorphism types of all maximal subgroups of *G* via IdGroup,
- *"SubId"*: the sizes and collected isomorphism types of conjugacy classes of all proper subgroups via IdGroup,
- *"MaxFactId"*: the collected isomorphism types of all maximal factors of *G* via IdGroup,
- *"CentQuot"*: the collected isomorphism types of all quotients of *G* with subgroups generated by non-trivial central elements of *G* via IdGroup,
- *"LCSFactId"*: the collected isomorphism types of all quotients of *G* with cyclic subgroups generated by non-trivial elements of the last non-trivial term of the p-central series,
- *"FactId"*: the collected isomorphism types of all proper factors of *G* via IdGroup,
- *"CocList"*: the collected numbers of conjugacy classes of *G* of same length and same representative order (the order of a representative),
- *"PMCycSt"*: the cycle-structure of the permutation induced by a power map specified by *opt.power* $x \mapsto x^n$ on the collected conjugacy classes (default $n = 3$),
- *"fpfunc"*: Uses the function provided as *opt.fpfunc*.

This function accepts a record *opt* of options consisting of the boolean *opt.pack*, that determines if the result of a test should be encoded as an integer, *opt.fpfunc* a costume function and *opt.power* the power used in the test "PMCycSt".

Example

```
gap> G := SmallGroup(512, 1405);;
gap> TestOnGroup(G, "ElmOrds");
85416
gap> TestOnGroup(G, "ElmOrds", rec(pack := false));
[ [ 1, 1 ], [ 2, 7 ], [ 4, 24 ], [ 8, 288 ], [ 16, 64 ], [ 32, 128 ] ]
```

2.4.4 TestOnClusterList

▷ `TestOnClusterList(CL, test[, opt])` (function)

Returns: Either a list of pairs each containing the path to the respective cluster (which is a leaf of a search tree) and the subbranch resulting from the results of *test*, or just the list of subbranches if *CL* contains only lists of integers. If *CL* consists of a single list of integers, the single resulting subbranch is returned on its own.

Performs the chosen test on the groups of each record in the *CL*. The tests are performed by `TestOnGroup` (2.4.3). *CL* could either be a list of records produced by `ClusterList` (2.4.2) or be a list of lists containing ids. In the former case, the output is a list of pairs containing the path of a cluster and the subbranch resulting from the test. In the latter case, only a list of subbranches is returned. If in the latter case, the returned list only contains one subbranch, the subbranch itself is returned. This is useful when building the first level of a search tree. This function accepts a record *opt* of options with the following content:

- *pack*: A boolean that is handed to `TestOnGroup` (2.4.3).
- *ign1cl*: A boolean that determines if clusters of size 1 should be ignored.
- *order*: An integer giving the order of the groups with ids in *CL*.
- *notetest*: A boolean that decides if the performed test should be noted in the returned subbranches. If so, subbranches that do not split a cluster are omitted.

Example

```
gap> TestOnClusterList([[1..10]],"ElmOrds");
rec( fp := [ 88570, 4310, 63049, 75721 ],
     next := [ [ 1 ], [ 2, 5, 6, 7, 9, 10 ], [ 3, 8 ], [ 4 ] ],
     test := "ElmOrds")
gap> TestOnClusterList([[1,2,5],[7,8,9]],"ElmOrds");
[ rec( fp := [ 88570, 4310 ], next := [ [ 1 ], [ 2, 5 ] ],
       test := "ElmOrds" ),
  rec( fp := [ 4310, 63049 ], next := [ [ 7, 9 ], [ 8 ] ],
       test := "ElmOrds") ]
gap> tree := TestOnClusterList([[3..8]],"ElmOrds");;
gap> CL := ClusterList(tree);
[ rec( descl := [  ], ids := [ 3, 8 ], path := [ 63049 ], size := 2,
       tests := [ "ElmOrds" ]),
  rec( descl := [  ], ids := [ 4 ], path := [ 75721 ], size := 1,
       tests := [ "ElmOrds" ]),
  rec( descl := [  ], ids := [ 5, 6, 7 ], path := [ 4310 ], size := 3,
       tests := [ "ElmOrds" ]) ]
gap> TestOnClusterList(CL, "MaxSubId", rec(notetest := false));
[ [ [ 63049 ], rec( fp := [ 62603 ], next := [ [ 3, 8 ] ] ) ],
  [ [ 4310 ], rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ] ) ] ]
```

2.4.5 AddClusterTestResults

▷ `AddClusterTestResults(tree, TR)` (function)

Returns: The extended *tree* with the subbranches in *TR*.

Extends *tree* by the test results stored in *TR*, a list of pairs, each containing the path in *tree* and a branch (a tree of depth one), extending the corresponding cluster of the initial *tree*.

Example

```
gap> tree := TestOnClusterList([[3..8]],"ElmOrds");
rec( fp := [ 63049, 75721, 4310 ], next := [ [ 3, 8 ], [ 4 ], [ 5, 6, 7 ] ],
     test := "ElmOrds")
gap> TR:=TestOnClusterList(ClusterList(tree),"MaxSubId");
[ [ [ 4310 ], rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ],
                   test := "MaxSubId" ) ] ]
gap> AddClusterTestResults(tree,TR);
rec( fp := [ 63049, 75721, 4310 ],
     next := [ rec( fp := [ 62603 ], next := [ [ 3, 8 ] ] ), [ 4 ],
               rec( fp := [ 11744, 86646 ], next := [ [ 5 ], [ 6, 7 ] ],
     test := "MaxSubId") ], test := "ElmOrds" )
```

2.4.6 BuildSearchTree

▷ `BuildSearchTree(`*N, order, tests*`)` (function)

Returns: A search tree (implemented as a record).

Builds a search tree for the groups with ids in *N* or, if *N* is an integer, the first *N* groups of size *order*, using `TestOnClusterList` (2.4.4) with the tests from the list *tests*.

Example

```
gap> BuildSearchTree(5,512,["ElmOrds", "MaxSubId"]);
rec( fp := [ 88570, 4310, 63049, 75721 ], next := [ [ 1 ],
     rec( fp := [ 76646, 11744 ], next := [ [ 2 ], [ 5 ] ],
     test := "MaxSubId" ), [ 3 ], [ 4 ] ], test := "ElmOrds" )
gap> BuildSearchTree([1,2,3,5,7], 512, ["ElmOrds", "CocList"]);
rec( fp := [ 88570, 4310, 63049 ], next := [ [ 1 ],
     rec( fp := [ 96561, 32173 ], next := [ [ 2 ], [ 5, 7 ] ],
     test := "CocList" ), [ 3 ] ], test := "ElmOrds" )
```

2.4.7 CocTestOnClusterList

▷ `CocTestOnClusterList(`*CL[, opt]*`)` (function)

Returns: A list of pairs each containing the path of the respective cluster and the subbranch resulting from `idpgrp_CocSplitRoutine`.

Tries to split each cluster in *CL* via tests on the lists of conjugacy classes. *CL* must be a cluster list produced by `ClusterList` (2.4.2). This function accepts a record *opt* of options with the following content:

- *pack*: A boolean that determines if the result should be encoded as an integer.
- *order*: An integer giving the order of the groups with ids in *CL*.
- *iterate*: An integer that determines the maximal number of iterations done while searching parameters.

Example

```
gap> tree := TestOnClusterList([[3,5,6,7,8]],"CocList");;
gap> CL := ClusterList(tree);
[ rec( descl := [  ], ids := [ 3, 8 ], path := [ 337 ], size := 2,
       tests := [ "CocList" ] ),
  rec( descl := [  ], ids := [ 5, 6, 7 ], path := [ 32173 ], size := 3,
```

```
        tests := [ "CocList" ] ) ]
gap> TR := CocTestOnClusterList(CL);;
gap> AddClusterTestResults(tree, TR);;
gap> ClusterList(tree);
[ rec( descl := [ [ 302004 ] ], ids := [ 3 ], path := [ 337, 1363 ], size := 1,
        tests := [ "CocList", "" ]),
  rec( descl := [ [ 302004 ] ], ids := [ 8 ], path := [ 337, 46760 ], size := 1,
        tests := [ "CocList", ""  ]),
  rec( descl := [ [ 104003 ] ], ids := [ 5 ], path := [ 32173, 1214 ],
        size := 1, tests := [ "CocList", ""  ] ),
  rec( descl := [ [ 104003 ], [ 40204005 ] ], ids := [ 6 ],
        path := [ 32173, 16, 20792 ], size := 1, tests := [ "CocList", ""  ] ),
  rec( descl := [ [ 104003 ], [ 40204005 ] ], ids := [ 7 ],
        path := [ 32173, 16, 46881 ], size := 1, tests := [ "CocList", ""  ] ) ]
```

2.4.8 ParametersRandPresentTest

▷ ParametersRandPresentTest(*CL*, *order*, *maxl*) (function)

Returns: A list consisting of pairs of a list of ids and a record containing the parameters for a random presentation test via ID_GROUP_FUNCS[18] defined in the SmallGrps package.

Determines the needed parameters for the groups of size *order* in the clusters *cl* of *CL*. Creates the clusters of conjugcy classes and alters them accordingly to the *cl.descl*. Afterwards minimal generators for all groups are chosen and their position among the clusters of conjugacy classes are determined. If the positions do not coincide, the positions of the generators of the first group are selected and the generators of the other groups are selected from the same clusters. Otherwise the positions and generators are sorted by the positions. Sequently the function tries to refine the corresponding clusters to limit the choices to choose from and repeat the process until either the number of refinments is equal or larger than *maxl* or no more refinments could be found.

Example

```
gap> ParametersRandPresentTest([rec(descl:=[], ids:=[1367, 1371])], 512, 0);
[ [ [ 1367, 1371 ], rec( descl := [  ],
     fp := [ 39080, 80814 ], func := 18, pos := [ 8, 10 ] ) ] ]
gap> ParametersRandPresentTest([rec(descl:=[], ids:=[1367, 1371])], 512, 1);
[ [ [ 1367, 1371 ], rec( descl := [ 106003 ],
     fp := [ 39080, 80814 ], func := 18, pos := [ 9, 11 ] ) ] ]
```

References

[BE99] H. U. Besche and B. Eick. Construction of finite groups. *Journal of Symbolic Computation*, 27(4):387–404, 1999. 5

[ES26] B. Eick and H. Schanze. The group identification problem for p-groups of small order. *Journal of Groups, Complexity, Cryptology*, 18:13, 2026. 3, 4

Index

AddClusterTestResults, 11

BuildSearchTree, 12

CalculateSiblings, 8
CalculateStrongSiblings, 8
ClusterList, 9
CocTestOnClusterList, 12

IdGroup512, 5
ID_GROUP_512_TREE, 6
InfoId512, 5
InfoSiblings, 8
InfoTreeBuild, 9

LoadTotalTree, 6

ParametersRandPresentTest, 13

Siblings, 7
StrongSiblings, 7

TestOnClusterList, 11
TestOnGroup, 9